tramdag documentation

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

> **Status: beta (0.x), under active development.** The API may change between releases until 1.0; pin a version (`tramdag==0.3.*`) for reproducibility.

TRAM-DAGs model each variable of a structural causal model with a (transformation-model) flow: one triangular normalizing flow from iid standard-logistic latents to the observed variables. The triangular adjacency structure is exactly your causal DAG. Fit it **once** on observational data and answer all three rungs of Pearl's causal hierarchy — observational (L1), interventional (L2, the do-operator), and counterfactual (L3, Pearl abduction) — while keeping **interpretable effects**: every linear-shift coefficient is a log-odds ratio, exactly as in classical proportional-odds models.

> Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 (arXiv:2503.16206). This repo is the reference implementation (PyTorch, built on zuko); all of the paper's experiments are replicated here with pinned tests.

5-minute showcase: the Colab badge above fits the paper's bimodal benchmark live (GPU-ready) and walks L1 → L2 → L3, every answer checked against analytic ground truth. Further notebooks are available at `notebooks/` like the didactic walkthrough of the model: `notebooks/intro_tram_dag.py`.

Install

```
pip install tramdag             # latest release (PyPI)
pip install "git+https://github.com/tensorchiefs/
        tramdag.git@main"   # dev version (track main)
uv sync                         # or: dev setup from a clone
        (tests, experiments)
```

Pin the dev install to a commit for reproducibility, e.g. `...tramdag.git@<sha>`.

30 seconds of API

```
import tramdag as td
from tramdag import CausalFlowDAG, ContinuousNode, OrdinalNode,
        I, LS, CS
```

```
spec = {  # the spec IS the labelled DAG
    "Age": ContinuousNode(),
    "mRS_pre": OrdinalNode(6, I("Age")),
    "NIHSSa": ContinuousNode(I("Age") + LS("mRS_pre")),
    "T": OrdinalNode(2, I("Age") + LS("mRS_pre") + CS("NIHSSa")),
    "mRS_3m": OrdinalNode(7, I("Age") + LS("mRS_pre") +
        CS("NIHSSa") + LS("T")),
}
flow = CausalFlowDAG(spec)  # validates acyclicity, builds the
        flow

# self-stopping training: per-node plateau lr decay + freezing of
        converged
# nodes (exact, since the per-node NLLs have independent
        gradients);
# see docs/training-speed.md for benchmarks and the classic two-
        phase recipe
flow.fit(
    train_df,
    val_df,
    epochs=4000,
    learning_rate=1e-2,
    batch_size=512,
    schedule="plateau",
    plateau_patience=30,
    freeze_patience=120,
)

# all-`ls` model? fit it classically instead: deterministic
        float64 L-BFGS,
# exact MLE matching statsmodels/R (see docs/fitting.md)
flow.fit_classical(train_df)  # raises on cs/ci specs

flow.log_prob(df)  # L1: joint log-likelihood per row
flow.sample(1000)  # L1: observational sampling
flow.sample(1000, do={"T": 1})  # L2: interventional (graph
        mutilation)
flow.pmf(df, node="mRS_3m", do={"T": 1})  # L2: analytic
        interventional PMF

u = flow.abduct(df)  # L3 step 1: latents from observations
cf = flow.sample(do={"T": 1}, u=u)  # L3 steps 2+3:
        counterfactuals

flow.ls_coefficients()  # interpret: per-edge log-odds-ratios
flow.intercept_contributions("NIHSSa", df)  # interpret: per-
        parent partial effects
```

```
# of an additive complex intercept (centered)

# heterogeneous treatment effects: a small, penalized effect head
        beta(x)*T
# (VC term) with a first-class read-out — see docs/varying-
        coefficients.md
# e.g. CS("Age", "NIHSSa") + VC("Age", t="T") ->
# flow.varying_coef("mRS_3m", df)          # beta(x):
        deterministic, y-free

flow.scores(df, node="mRS_3m")  # per-observation scores dl_i/
        dtheta
flow.effect_modifier_scan(df, "mRS_3m", t="T")  # which VC
        modifiers? (CUSUM
# scan from a cheap all-ls fit) — docs/scores.md

flow.save("flow.pt")
flow = CausalFlowDAG.load("flow.pt")

td.simulations.REGISTRY  # synthetic DGPs with known ground truth
```

The model in detail: spec → math → networks

Per node, the transformation is additive on the latent (log-odds) scale — one intercept term `I` plus any number of shifts (notation: docs/notation.md):

`u = h_ϑ(x) + Σ β·x_pa + Σ g(x_pa) + (β₀ + b_θ(x_mod))·x_t`

term	math	what gets built	interpretability
`I()` / bare `I` / omitted	`h_ϑ(x)` — constant ϑ	`SimpleIntercept`: one free parameter vector, no network	the baseline transform
`I("A")`	`h_ϑ(a)(x)` — ϑ bends with the parent	`ComplexIntercept`: MLP `[8, 8]` → `n_params`	the parent reshapes th distribution; no single
`I("A","B")` (default `allow_interaction=True`)	`h_ϑ(a,b)(x)`	**one joint** MLP over both parents — they interact in ϑ	maximal flexibility
`I("A","B", allow_interaction=False)`	`h_ϑ(a)+ϑ(b)(x)`	one MLP **per parent**, parameter vectors summed in coefficient space	per-parent partial effe `flow.intercept_cont`
`LS("A")`	`β·a`	`Linear(width, 1)`, no bias — **one parameter**	`exp(β)` is an odds rati
`CS("A")`	`g(a)`, additive		plot `g`

term	math	what gets built	interpretability
		`ComplexShift`: MLP `[64, 128, 64] → 1`	
`CS("A","B")`	`g(a,b)` — joint	one MLP over the concatenated features	interaction *in the shift*
`CS("A") + CS("B")`	$g_1(a) + g_2(b)$	two MLPs, scalars added	GAM-style, each effect
`VC("A","B", t="T")`	$(\beta_0 +$ `b_θ(a,b))·x_t`	scalar β_0 + zero-initialised penalized MLP `[16] → 1`; the treatment value multiplies, it never enters the net	β_0 ≈ constant effect, `flow.varying_coef` re

A node takes **at most one `I` term with parents** — a term list is therefore always purely additive on the latent scale, and interactions exist only *inside* a term. Lists and `+` sums are interchangeable: `[LS("A"), CS("B")]` ≡ `LS("A") + CS("B")`.

Two knobs on the terms:

- **`transform=` on `I`** picks the basis of `h_ϑ` for a continuous node — `"bernstein"` (default, 20 coefficients, tails extrapolate with the boundary slope), `"spline"` (monotone RQ spline, 23 params at `bins=8`, fixed tail slope) or `"affine"` (2 params: the latent is exactly logistic). Ordinal nodes have no basis: their intercept is the cutpoint vector, `P(x ≤ k) = σ(ϑ_k − shift)`.
- **`units=` on `I`/`CS`/`VC`** sizes the term's network, e.g. `units=[16]` for one hidden layer of 16 neurons (defaults above match the original Keras implementation, so fits stay comparable).

Feature widths: a continuous parent enters raw (1 column), an ordinal parent one-hot (`levels` columns). Abduction is exact for continuous nodes and truncated-logistic for ordinal ones, so `flow.sample(u=flow.abduct(df))` reproduces `df` exactly / level-exactly.

There are two ways to fit the model: a stochastic deep learning optimizer (`fit`) and a 2nd order optimization like in the classical statistical models (`fit_classical`). The latter is more efficient for all-`ls` models (each node-conditional is then a classical transformation model). For more details, see the docs/fitting.md file.

Validation (all pinned by tests)

- **Paper replication** — each of the paper's four DGP families has a generator in the registry (numpy-only SCM + frozen CSVs + replication script) and is pinned by tests:

family	paper	demonstrates
`triangle` (`linear,atan,sin`)	§6.1	LS coefficient recovery (β = 2, −0.2, +0.3), CS curve ≡ −f(x₂), non-monotone f
`triangle-mixed` (`linear,exp`)	§6.2	mixed data L1/L2 + the C.4 odds-ratio check (OR ≈ 7.4)
`vaca`	§5.1–5.2	the bimodal L1 case a default CNF misses; L2 `p(x₃ | do(x₂))`
`carefl`	§5.3	L3 counterfactual curves vs **analytic** truth

Sign note: ordinal shifts are *subtracted* here but *added* in the paper, so fitted ordinal weights are the paper's with flipped sign (`truth.json` records both conventions per family).

- **Exact classical equivalence** — an all-`ls` flow trained to convergence *is* the proportional-odds MLE: coefficients match `statsmodels` **and** R `MASS::polr` to ~4 decimals (`experiments/validate_ls.py`, R reference committed under `data/magic-mrclean/*/ref_ls/`).
- **Training speed** — schedules, per-node freezing, LBFGS and device benchmarks: docs/training-speed.md.

What the tests actually guarantee — the principles behind them (known identities, the datasets and software they compare against, and how the ground truth was obtained) — is documented in tests/README.md.

Full storyline, clinical-data context, R cross-check and reading notes: docs/stroke-case-study.md.

Testing policy

See the tests/README.md file for more details.

Layout

```
src/tramdag/            spec.py transforms.py conditioners.py
flow.py
                        scores.py env.py
                        simulations/   (magic_mrclean, triangle,
vaca, carefl,
                                        vc_shift + CLIs)
data/                   frozen synthetic CSVs + truth.json — a
test contract
experiments/            stroke pipeline + paper replications
notebooks/              intro (didactic) + Colab demo
(jupytext .py — see README there)
tests/                  unit, known-truth recovery, R regression
docs/                   code-map.md (every class/function + all
knobs),
```

```
                          fitting.md, notation.md, training-
speed.md,
                          stroke-case-study.md, varying-
coefficients.md, scores.md
```

Implementation conventions (latent-scale signs, raw/one-hot parent encoding, log-space ordinal likelihood, seeding) are documented in CLAUDE.md and pinned by tests.

Citation

If you use `tramdag`, please cite the method paper:

```bibtex
@inproceedings{sick2025tramdag,
  title     = {Interpretable Neural Causal Models with TRAM-
       DAGs},
  author    = {Sick, Beate and D{\"u}rr, Oliver},
  booktitle = {Proceedings of the 4th Conference on Causal
       Learning and Reasoning (CLeaR)},
  series    = {Proceedings of Machine Learning Research},
  volume    = {275},
  year      = {2025},
}
```

Code map — every class and function in `src/tramdag/`

One entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery: useful to know, not part of the API. The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term constructor has a pythonic name and a short alias; the two are the same object, so `LS is linear_shift`.

Name	Role
`Term`	One additive term of a node's transformation: the frozen triple `(effect, parents, options)`. `+` on terms builds plain lists. Effect-specific settings live in `options` and read as attributes (`term.penalty`, `term.units`, ...).
`intercept()` / `I`	Intercept term: the parents reshape the monotone transform. `I()` or the bare name `I` is the simple-intercept baseline. Carries the basis choice (`transform=`, default

Name	Role
	"bernstein") and allow_interaction= (joint vs. additive multi-parent intercept).
linear_shift() / LS	Linear shift beta * x — the interpretable log-odds coefficient. Exactly one parent.
complex_shift() / CS	Complex shift: an MLP g(x), additive on the latent scale. Several parents form one joint network.
varying_coefficient() / VC	Varying-coefficient shift (beta0 + b_theta(mods)) * x_t — the penalized treatment-effect head (issue #28). center= adds propensity centering (issue #30).
ContinuousNode	Continuous variable: monotone 1-D transform plus shifts. terms is the first positional argument.
OrdinalNode	Ordinal variable with levels classes: ordered logit (cutpoints) plus shifts.
node_terms() / node_parents()	Canonical term list / ordered de-duplicated parent names of a node.
validate_and_sort()	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
spec_to_dict() / spec_from_dict()	Checkpoint (de)serialization. A term serializes as {effect, parents, options} and nothing else, since options is already canonical. No compatibility shims, and spec_from_dict builds Term directly — so validate_and_sort is the only guard on that path.
(_normalize_terms, _as_term, _check_intercepts, _options, _OPTION_DEFAULTS)	Formula flattening and per-entry validation (a + sum nested in a list is rejected), the one-parented-I rule plus basis hoisting in one pass, canonical option storage.

transforms.py — the monotone map h and the ordinal transform

Name	Role
StandardLogistic	The TRAM base distribution: log_prob, sample (generator-aware), icdf.
BernsteinUT	Bernstein-polynomial transform (default basis, n_coeffs=20). Linear tail extrapolation follows the boundary derivative. marginal_init_theta() gives the calibrated start used by fit(marginal_init=True).
SplineUT	

Name	Role
	Monotone rational-quadratic spline (`bins=8`). Tails extrapolate with a *fixed* slope — the structural reason spline trails Bernstein on tail-heavy data.
`AffineUT`	Monotone affine transform: the node-conditional is a logistic GLM.
`make_univariate_transform()`	Basis registry: name → transform instance.
`ordinal_cutpoints()`	Unconstrained `(n, K-1)` → increasing cutpoints with ±inf ends. Port of the original parametrization.
`ordinal_log_prob()`	`log P(Y=y)`, computed in log-space. Load-bearing: the naive sigmoid difference saturates in float32 and freezes nodes at init. Do not simplify.
`ordinal_pmf()` / `ordinal_sample()` / `ordinal_abduct()`	Class probabilities / latent → level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
`ordinal_marginal_init_theta()`	Cutpoint start that matches the empirical class frequencies (`marginal_init`).
`(_ScaledUT, _expanding_bisection, _bounds, _log1mexp)`	Quantile pre-scaling base class; tail-safe inverse; per-level cutpoint intervals; stable `log(1-exp(x))`.

`conditioners.py` — the networks behind the terms

Architectures replicate the original Keras implementation, so fitted models stay comparable to it.

Name	Term	Role
`SimpleIntercept`	bare `I`	Free parameter vector; no parents.
`ComplexIntercept`	`I(...)`	8-8 ReLU MLP from parent features to the transform parameters.
`LinearShift`	`LS`	`Linear(n, 1, bias=False)`. `.weight` is the interpretable coefficient; no bias because the intercept slot owns the constant.
`ComplexShift`	`CS`	64-128-64 ReLU MLP to one shift value.
`VaryingCoef`	`VC`	`beta0 + b_theta(mods)` with a zero-initialized output layer and the L2 hook `l2()`. `beta()` evaluates the effect, `recenter()` re-splits `beta0`/`b_theta` after training (function-preserving).
`(_mlp)`	—	The one MLP builder: a ReLU stack of the given `units`, then a bias-free output layer.

`flow.py` — the model

Name	Role
`CausalFlowDAG`	The flow: one `_Node` per variable in topological order. Construction seeds the weights (`seed=` is the reproducibility knob).
`fit()`	Joint maximum likelihood with Adam, one parameter group per node (exact, because the NLL decomposes per node). Options: `schedule="plateau"` (per-node decay), per-node freezing, `restore_best`, `marginal_init`, `vc_warm_start`, `plateau_factor`, `vc_oof_fit`. A second call continues training. Progress goes to the `tramdag.flow` logger.
`fit_classical()`	Float64 full-batch L-BFGS for all-`ls` specs: deterministic, exact MLE, matches `statsmodels`/R `polr`. Refuses flexible specs.
`sample()`	Observational, interventional (`do=`, graph mutilation) and counterfactual (`u=`) sampling.
`abduct()`	Pearl step 1: recover the latents. Continuous exactly, ordinal by truncated draw.
`pmf()`	Analytic class probabilities of an ordinal node, with `do=` overrides.
`log_prob()` / `nll()`	Joint per-row log-likelihood / mean per-node NLL diagnostic.
`node_log_prob()`	The per-node decomposition everything trains and evaluates through.
`varying_coef()`	Closed-form read-out `beta(x)` of a fitted VC term. Deterministic, y-free.
`scores()` / `effect_modifier_scan()`	Analytic per-observation scores and the CUSUM modifier scan (delegate to `scores.py`).
`intercept_contributions()`	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
`ls_coefficients()`	The per-node linear-shift weights — the interpretable coefficients.
`to_matrix()`	The labeled meta-adjacency matrix of term effects.
`save()` / `load()`	Checkpoints with history and machine provenance. `load` requires a complete checkpoint and fails loudly otherwise.

Name	Role
(_Node, _VCGroup)	Per-node module (intercept + shift ModuleDict + VC bookkeeping); theta_shift() computes (theta, shift).
(_node, _encode_parent, _features, _tensorize, _dtype, _np_dtype)	Node lookup with one shared error; parent encoding (continuous raw, ordinal one-hot); _tensorize(df, cols=None) for any column subset; dtype plumbing.
(_set_ranges)	Train 5%/95% quantiles onto the transform domain, plus the optional marginal initialization. First fit only.
(_vc_warm_start, _vc_oof_stage, _vc_oof_propensity, _predict_p1, _vc_ehat_live, _vc_ehat_columns, _recenter_vc, _source_proxies)	The VC machinery: classical beta0 warm start; frozen out-of-fold propensities for training (DML); live full-fit propensities for inference (recomputed under do, never cached); post-fit re-centering.
(_is_all_ls)	Guard for fit_classical.

scores.py — effect-modifier detection (issue #29)

Name	Role
node_scores()	Analytic, exact per-observation scores psi_i = d l_i / d theta for every LS weight and VC beta0. No autograd.
effect_modifier_scan()	Zeileis-Hornik fluctuation scan: order the treatment scores by each candidate, `sup
sup_bb_pvalue()	`P(sup
(_dl_ds, CRIT_5PCT)	Closed-form latent-scale derivative; the 5% critical value 1.3581.

env.py

Name	Role
machine_info()	Machine/software snapshot stored by save(), so timings stay comparable across machines. Never raises.

simulations/ — numpy-only ground truth

Each generator is independent of the flow implementation and has a CLI that regenerates its frozen data/<name>/ CSVs (a test contract — never regenerate silently). REGISTRY maps name → class.

Scheduled to move to the simulation-study companion repo (`tensorchiefs/tramdag-simu`) with `experiments/` and the data-contract tests; frozen until then.

`_common.py` holds what the generators share: `logistic`, `sigmoid`, `resolve_latents` (the `n`/`rng`/`latents` triple), and the `DatasetDraws` mixin — `observational`, `interventional`, `counterfactual_pair`. Those three carry the seed offsets (`+1`, `+501`, `+2`) that the frozen CSVs in `data/` depend on, so they are defined once. Each generator module then holds only its own structural equations and ground truth.

Class (module)	DGP	Ground-truth read-outs
`MagicMrClean` (`magic_mrclean`)	Synthetic stroke cohort, `ls`/`nl` variants	`true_ate()`, `counterfactual_pair()`, `observational()`, `rct()`
`TriangleContinuous` / `TriangleMixed` (`triangle`)	Paper §6 triangles, f variants linear/ cubic/exp/atan/ sin	`paper_truth()`, `zuko_expectations()`, `true_shift_curve()`, `true_pmf()` (mixed), `interventional()`, `counterfactual_pair()`
`VacaTriangle` (`vaca`)	App. C.1 bimodal Gaussian benchmark	`true_moments()` (analytic do-moments), `interventional()`
`Carefl4` (`carefl`)	App. C.2 Laplace SCM	`abduct_noise()`, `true_counterfactual()`, `true_cf_curves()` — all analytic
`VCLogisticShift` (`vc_shift`)	Issue #28 heterogeneous-effect DGP	`true_beta()` (known effect function), `counterfactual_pair()`

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
epochs, learning rate, batch size	`fit()`	500 / 1e-2 / 512 (in-repo examples always state them explicitly)
schedule, plateau patience, plateau decay factor	`fit(schedule=, plateau_patience=, plateau_factor=)`	None / 15 / 0.3 (lr floor `1e-3 * learning_rate`, stated in the docstring)
per-node freezing	`fit(freeze_patience=, min_delta=)`	off / 1e-4 (freeze guard `1e-2 * learning_rate`)
	`fit(restore_best=)`	False = exact MLE

Knob	Where	Default
early stopping		
calibrated init	`fit(marginal_init=)`	False (pure init, MLE unchanged)
VC warm start	`fit(vc_warm_start=)`	True (classical `beta0` start)
VC stage-1 proxy fits	`fit(vc_oof_fit=)`	`{"epochs": 300, "learning_rate": 1e-2, "batch_size": 512}`
VC penalty and centering	`VC(penalty=, center=, center_folds=)`	1.0 / False / 5
L-BFGS budget	`fit_classical(max_iter=, tol=, chunk=, history_size=)`	400 / 1e-6 / 25 / 50
network widths	`units=` on `I`/`CS`/`VC`	(8, 8) / (64, 128, 64) / (16,) — Keras-parity architecture defaults
transform basis	`I(transform=, **kwargs)` (extra kwargs go to the transform class)	`"bernstein"`, `n_coeffs=20`; spline `bins=8` (the domain is fixed at [-5, 5], `transforms.BOUND`)
shuffling / weight init	`fit(seed=)` / `CausalFlowDAG(seed=)`	init happens at construction — the constructor seed is the reproducibility knob

Fitting a TRAM-DAG: how training works

This page is the technical reference for `CausalFlowDAG`. It explains how the module builds the flow and how it computes the likelihood. It describes the two fitting paths: the stochastic optimizer (`fit`) and the classical optimizer (`fit_classical`). It also covers per-node freezing, the memory model, and future directions for optimizer choice.

How the flow is built: one module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module`, but it is **not one monolithic network**. At construction (`CausalFlowDAG.__init__`), it topologically sorts the DAG. It then builds an `nn.ModuleDict` with **one `_Node` sub-module per**

variable. The nodes share *no parameters*. Each node owns the pieces for its own conditional `p(x_i | pa(x_i))`:

- an **intercept** that produces the transform parameters `θ`. This is `SimpleIntercept`, a free parameter vector that is data-independent. If the node has `ci` parents, it is `ComplexIntercept` instead, a small MLP whose output `θ` depends on those parents.
- a **monotone 1-D transform** `h` (`BernsteinUT / SplineUT / AffineUT`). The transform itself carries **no learnable weights**, only the fitted range buffers `xmin`/`xmax`. The `θ` from the intercept sets its shape entirely.
- a `ModuleDict` of **shift modules**, one per parent edge: `LinearShift` (`ls`, a single weight) or `ComplexShift` (`cs`, an MLP).

So is this one network or several? It is **one module, several independent per-node sub-models** (each itself a small intercept + shifts assembly). One module bundles them, and one optimizer trains them, but their parameters are disjoint. The DAG structure lives entirely in *which parents each node reads*. There is no edge weight matrix or shared trunk.

For a batch, `_Node.theta_shift` assembles a node's `θ` (from the intercept) and total `shift` (sum of its shift modules over the parent features). `_encode_parent` encodes the parent features: continuous parents raw, ordinal parents one-hot.

How the likelihood is computed

A TRAM-DAG maps iid standard-logistic latents `U` to the observed `X` in causal order. Node `i` reads only its parents (earlier variables, as *data*). Therefore the Jacobian of `U → X` is **triangular**, and its log-determinant is the sum of the per-node 1-D terms. The joint log-likelihood therefore **decomposes per node**:

```
log p(x) = Σ_i log p(x_i | pa(x_i))
```

`CausalFlowDAG.node_log_prob` computes one term per node and `log_prob` sums them. For a node, given `θ, shift` from `theta_shift`:

- **continuous** — change of variables through the monotone transform:

  ```
  z = h(x; θ) + shift
  log p(x | pa) = log f_logistic(z) + log |dz/dx|
  ```

  That is, the term is the standard-logistic density at the latent `z` (`StandardLogistic.log_prob`) plus the transform's log-derivative. `ut.forward` returns this log-derivative as `ladj`. This is the 1-D Jacobian term that makes the result a proper density, not only a score.

- **ordinal** — an ordered-logit / proportional-odds head, `P(x ≤ k) = σ(θ_k − shift)`. `ordinal_log_prob` evaluates it as the log of the cutpoint-interval probability. The computation runs in log-space via

`logsigmoid`/`log1mexp`, because the naive sigmoid difference underflows to exactly-zero gradients in float32.

The training loss is the summed per-node **mean** NLL over the batch (`Σ_i mean_rows(-log p(x_i | pa))`). In contrast, `log_prob` returns the per-row joint, which scores whole observations.

Consequence used by both optimizers: because parents enter as data, the per-node gradients are independent. Therefore a joint fit of the summed loss is identical to a separate fit of each node. This independence licenses per-node learning rates, per-node freezing (below), and the all-`ls` classical fit.

Path A — stochastic optimization (`fit`)

`CausalFlowDAG.fit` is the general-purpose trainer. Any `cs`/`ci` edge requires it. Mechanics:

- **Adam** with **one parameter group per node**. `fit` builds each group from `self.nodes[name].parameters()`, so each node can carry its own learning rate.
- **Minibatches**: a fresh `torch.randperm` shuffle occurs each epoch. The loss is the summed per-node mean NLL on the batch.
- **Transform ranges**: `_set_ranges` sets them once at the start from the train 5%/95% quantiles. These ranges define each Bernstein/spline domain.
- **Learning-rate schedules** (`schedule=`): `None` (constant) or `"plateau"`, which decays each node's lr off *its own* validation NLL. (`"onecycle"` and `"cosine"` lost to plateau in the June 2026 benchmark and were removed in 0.4.0.)
- **Per-node freezing** (`freeze_patience=`, see below).
- **`restore_best`**: when enabled, `fit` snapshots each node's best-validation weights and restores them at the end (early-stopping regularization). The default is `False`, so the fit sits at the training-data MLE.
- **`marginal_init=`**: opt-in calibrated initialization of the unconditional intercepts — Bernstein nodes start at the linear map onto the latent 5%/95% quantiles, ordinal cutpoints at the empirical class log-odds. A pure init; the converged MLE is unchanged.
- **`vc_warm_start=`** (default on): each `VC` term's `beta0` starts at the classical all-`ls` solution of its node's conditional, so the penalized head only learns deviations.
- **`plateau_factor=`** (default 0.3): the multiplier of one per-node plateau decay step. Read only under `schedule="plateau"`; the floor is `1e-3 × learning_rate`, and under this schedule a node may only freeze once its lr has decayed to `1e-2 × learning_rate`.
- **`vc_oof_fit=`**: keyword overrides for the stage-1 out-of-fold propensity fits behind `VC(center=True)`, merged over `{"epochs": 300, "learning_rate": 1e-2, "batch_size": 512}`. Ignored when the treatment node is all-`ls`, which takes the deterministic `fit_classical` path instead.

How per-node freezing works

Set `freeze_patience=N` to remove converged nodes from training. Each epoch, `fit` tracks every node's own validation NLL and counts epochs since its last improvement (by more than `min_delta`). When that counter reaches `N`, `fit` **freezes** the node. Under `schedule="plateau"`, the freeze occurs only after the schedule decays the node's learning rate. This order prevents a freeze while a smaller step can still help.

A freeze does more than stop updates. `fit` **excludes the frozen node from the computation entirely**: it calls `node_log_prob(values, nodes=active)` with only the still-active nodes, so a frozen node runs no forward and no backward pass. That is a real FLOP saving. It is also *correct*, precisely because the per-node gradients are independent (above): removal of a converged node's term from the loss does not change any other node's gradient. The slowest node sets the total training time. Once the other nodes converge, they cost nothing.

When **every** node is frozen, the fit returns early (the self-stopping behavior). `fit` records the freeze epochs in `flow.history["frozen"]`. The freeze state is local to a single `fit` call (a later `fit` call trains all nodes again). This composes with `restore_best`, which still restores each node's best-validation snapshot at the end.

Freezing vs. parallelism (why one helps and the other usually does not)

A tempting argument says: "freezing a node saves time, so node computation costs time, so parallel computation of nodes will also save time." The premise is correct, but the conclusion does not follow. The two act through different mechanisms:

- **Freezing removes work.** The frozen node's forward/backward never runs again. Removal of work is an *unconditional* win, and most of its payoff is structural. Once *all* nodes freeze, the fit stops and deletes whole epochs. Parallelism can never do that (it can shrink time-per-epoch, not the number of epochs).
- **Parallelism only overlaps work.** It runs the same computations concurrently, and this helps *only if the hardware sits idle* during the sequential node loop. The hardware usually is not idle. Parallelism already exists one level down: each node's batched tensor ops run across all rows, and the BLAS/GPU backend already uses all cores. While node A's matmul runs, the cores are busy, so node B "at the same time" only time-slices the same cores. The result is contention, often slightly slower.

So the speed gain from freezing does **not** imply spare capacity for parallel work across nodes. The exception is the regime where per-node ops *under-utilize* the hardware: small batches, tiny models, or many small nodes on a big GPU where each kernel leaves it mostly idle. In that regime, node overlap can help. But the right tool is **fusion** (batch same-shaped nodes into one larger tensor op, or CUDA streams), not a threaded Python loop (which

the GIL fights). That is the "vectorize/fuse the per-node loop" direction in Optimizer choice below. It is a conditional win in that regime, distinct from the unconditional win of freezing.

Benchmarks, schedule trade-offs, and the recommended self-stopping recipe are in training-speed.md. The worked walkthrough is `notebooks/intro_tram_dag.py`.

Path B — classical optimization (`fit_classical`)

`CausalFlowDAG.fit_classical` is the dedicated optimizer for **all-`ls`** models, where every node-conditional is a classical transformation model (ordered-logit / Colr). It raises on any `cs`/`ci` edge.

- **Full-batch, float64, L-BFGS** (strong-Wolfe line search). There are no minibatches, no schedule, and no early stopping. Therefore the fit is **deterministic** (same init → bit-identical) and lands on the **exact MLE** (matches `statsmodels`/R to ~1e-3 on well-identified coefficients).
- **Solver budget** (`max_iter=400`, `tol=1e-6`, `chunk=25`, `history_size=50`): the fit runs in rounds of `chunk` inner L-BFGS iterations and checks NLL flatness between them, so `n_iter` in the report counts in multiples of `chunk`. `history_size` is the L-BFGS memory.
- **float64 is a transient compute mode**: `self.double()` upcasts parameters *and* the transforms' range buffers in one call. The fit runs in double. A `finally: self.float()` restores float32. The stored model stays float32, and so do `save`/`load`. The data path (`_tensorize`, `sample`, `pmf`) reads the model dtype, so it follows along.
- **Convergence**: the flag comes from NLL flatness and is *advisory*. A Bernstein intercept and weakly-identified directions (rare one-hot levels, a flat treatment-effect ridge) continue to drift along zero-curvature valleys after the likelihood is at the optimum. Correctness comes from a comparison with classical software (`experiments/validate_ls.py --classical`), not from the flag.
- Read the fitted coefficients with `ls_coefficients()`.

Walkthrough: `notebooks/stale/classical_fit_tram_dag.py` (parked).

Memory and disk during fitting

Neither fitting path writes to disk. Everything during `fit`/`fit_classical` lives in RAM:

- model parameters and (for `fit`) the per-node Adam optimizer state
- the `history` dict (per-epoch train/val NLL, lr, wall-clock), which is an in-memory attribute and not a file
- `restore_best` snapshots: `copy.deepcopy` of node `state_dicts` held in memory (`flow.py`), never serialized

The **only** disk I/O in the module is the explicit, user-called save (`torch.save` of spec + state_dict + history) and its counterpart `load`. So a fit produces no temp files, no checkpoints, and no scratch directory. Persistence is opt-in via `flow.save(path)`. (The *experiment scripts* write the `results/` and `docs/perf/` artifacts in this repo, not the library's fitting code.)

Optimizer choice — current and future

Today: **Adam** for flexible models, **L-BFGS** (float64) for all-`ls`. The per-node-param-group structure already in `fit` makes several extensions cheap. These extensions are worth consideration as the package matures:

- **IRLS / Fisher scoring for the all-`ls` path.** The classical fitting algorithm for proportional-odds / GLM-type models is iteratively reweighted least squares (Newton with the expected information). For the `ls` case, it will likely reach the MLE in *fewer, more stable* steps than L-BFGS. Also, the Fisher information that it computes is exactly the ingredient for the planned **standard-error table** (currently flagged as next in the CHANGELOG). It is a strong candidate to back `fit_classical` in a future version.
- **Second-order / curvature-aware methods for flexible nodes.** K-FAC or a Gauss-Newton approximation can help the `ci`/`cs` MLPs. However, full-batch quasi-Newton is a poor fit for them (minibatch noise also regularizes, which is why `fit_classical` refuses them).
- **Modern first-order variants.** AdamW (decoupled weight decay), RAdam (warmup- free), or Lion/Sophia are drop-in alternatives to Adam. The benchmark harness (`experiments/stale/bench_training.py`, parked — needs an API migration) exists to evaluate exactly such swaps on time-to-target.
- **Per-node optimizer selection.** The loss decomposes, and the optimizer already holds one group per node. Therefore different nodes can in principle use different optimizers (for example, L-BFGS for the `ls` nodes and Adam for an MLP node in a mixed model). This is a direction for mixed-flexibility DAGs.
- **Warm-start handoffs.** `fit_classical → fit` already works (the classical MLE as a fast, principled initialization). The reverse (Adam to escape, L-BFGS to polish) is a natural pattern to formalize.

These are *directions*, not commitments. The autoresearch loop (`docs/research/MISSION_autoresearch.md`) is a good venue to test the speed-oriented ones empirically before any adoption.

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y. Their realizations — observed or sampled values — are lowercase Latin letters, for example y. The cumulative distribution function of Y is F_Y and the density is f_Y. A hat marks an estimate: $\hat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, ..., X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β. A vector of parameters is bold lowercase, for example $\boldsymbol{\vartheta} = (\vartheta_1, ..., \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example $\boldsymbol{\Theta}$.

A function that a parameter vector defines carries it as a subscript: $h_{\boldsymbol{\vartheta}}$ is the monotone transformation with coefficients $\boldsymbol{\vartheta}$.

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\mathrm{pa}(X_j)$	the causal parents of X_j	`node_parents`
U_j	the standard-logistic latent of node j	`u = flow.abduct(df)`
$h_{\boldsymbol{\vartheta}}$	the monotone transformation of a node	the `I` term; `theta` in docstrings
ϑ_k	the ordinal cutpoints, elements of $\boldsymbol{\vartheta}$	`ordinal_cutpoints`
β	a linear-shift coefficient; e^β is an odds ratio	`LS`; `flow.ls_coefficients()`
g_j	a complex shift, an MLP of one term's parents	`CS`
β_0	the constant treatment effect of a `VC` term	`beta0`
$b_{\boldsymbol{\Theta}}$	the penalized effect-modification network	the `VC` net; `units=` sets its layers
λ	the L2 weight on $b_{\boldsymbol{\Theta}}$	`penalty=`
σ	the logistic function	`torch.sigmoid`

Every node's model is one additive formula on the latent scale:

$$u = h(x \mid \mathrm{pa}(x)) = h_{\boldsymbol{\vartheta}(\cdot)}(x) + \sum_j \beta_j x_j + \sum_k g_k(x_k) + (\beta_0 + b_{\boldsymbol{\Theta}}(x_{\mathrm{mod}}))\, x_t.$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \le k \mid \mathrm{pa}) = \sigma(\vartheta_k - \mathrm{shift})$.

Plain-text fallback

Docstrings and terminal output cannot render Greek subscripts. There, `theta` stands for $\boldsymbol{\vartheta}$, `h_theta` for $h_{\boldsymbol{\vartheta}}$, `b_theta` for $b_{\boldsymbol{\Theta}}$ and `beta0` for β_0.

Per-observation scores & the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation for the node's interpretable shift coefficients. These coefficients are every `LS` weight and every `VC` term's `beta0`. An `LS` weight gives one column per continuous parent and one column per one-hot level of an ordinal parent. The `beta0` column is named after the treatment.

The computation is **analytic and exact**, with no autograd. Shifts enter the latent additively. Thus $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) \cdot x_i$, with $\partial \ell_i / \partial s$ in closed form (`src/tramdag/scores.py`). The function is a pure read-out. It does not touch the fitting or sampling code paths. At a fitted MLE, the sum of each column is ≈ 0. Tests pin this behavior and include a float64 finite-difference check.

Worked example: where does β(x) need to bend?

The score is the cheapest effect-modifier detector available. It uses the structural-change logic of model-based recursive partitioning (Zeileis & Hornik; Dandl et al. 2024). Before you fit an expensive model, fit the **cheap all-LS model**. This fit takes seconds and is deterministic. Then scan the treatment-coefficient scores:

```
flow = td.CausalFlowDAG(all_ls_spec, seed=0)
flow.fit_classical(df)  # seconds, exact MLE

scan = flow.effect_modifier_scan(df, node="Y", t="T")
#         stat   p_value  crit_5pct   flag
# X2      4.21    <0.001     1.3581   True     <- true modifier
# X3      3.05    <0.001     1.3581   True     <- true modifier
# X1      0.74     0.64      1.3581  False     <- prognostic only
```

For each candidate covariate, the scan orders the scores by that covariate and forms the scaled cumulative sum `B_j = Σ_{i≤j} ψ_(i) / (sd(ψ)·√n)`. Under a stable coefficient, `B` is a Brownian bridge. Thus `sup|B|` has the Kolmogorov distribution (5% critical value 1.358). A coefficient that truly varies with the covariate makes the ordered scores drift. The flagged covariates are the measured shortlist of `VC` modifiers (`docs/varying-coefficients.md`):

```
spec["Y"] = td.ContinuousNode(
    td.CS("X1", "X2", "X3") + td.VC("X2", "X3", t="T")
)  # scan-informed
```

Notes: For a binary ordinal `LS` treatment, `t` resolves to the identified level-1-vs-0 contrast. For a `VC` treatment, `t` resolves to `beta0`. Candidates default to every column of `df` except the node and `t`. Modifiers do not need to be parents. For few-level (heavily tied) candidates, the ordering is only partial. Then read the scan as a ranking diagnostic, not as an exact-size test. You can also use the scores later for influence-function analyses and robust standard errors.

Case study: the stroke ITE analysis

This document gives the background and the full detail for the `magic-mrclean` experiments. These experiments are the public, synthetic counterpart of the clinical analysis in Dürr, Herzog, Bühler, Wegener & Sick, *Estimating Individualized Treatment Effects in Acute Ischemic Stroke with Causal Transformation Models (TRAM-DAG)* (arXiv:2606.12623). The clinical MAGIC / MR CLEAN data is private. It is never part of this repo.

The DAG and the pipeline

The DAG has five nodes with all forward edges:
`Age → mRS_pre → NIHSSa → T → mRS_3m`. In the observational cohort, age and stroke severity confound the treatment `T` (thrombectomy). The estimand is the ATE of `T` on a good outcome, `P(mRS_3m ≤ 2 | do(T))`, averaged over the (younger) trial population. The pipeline computes this ATE analytically from the interventional PMF of the outcome node. It does not use Monte Carlo.

```
cd experiments
uv run python sim_flow.py nl                    # headline
        storyline: all-ls vs

        # flexible vs known truth
uv run python validate_ls.py                    # all-ls flow vs
        statsmodels MLE
# parked in stale/ (need an API migration before use): the
        single-config
# runners (all_ls_flow, nihss6_flow), counterfactual_demo.py, and
        the
# 4x-longer convergence check all_ls_long.py
```

Experiments default to the public synthetic source `magic-mrclean/nl`. The `magic` source (clinical cohort, NIHSSa ≥ 6, N = 1275) works only inside the original paper monorepo. Every run uses the same 80/10/10 split

(`random_state=42`). Each run writes plots, per-patient interventional PMFs (`rct_predicted_proba.csv`), and a checkpoint to `results/<name>/`.

The synthetic cohort (`magic-mrclean`)

This cohort is a hand-specified SCM in the model family of the flow itself (logistic latents). It has the same schema as the stroke study and realistic marginals. It also has **known ground truth** that the real data cannot provide: the true ATE, true individual counterfactuals (shared-latent pairs), and a provably misspecified baseline. The cohort has two variants:

- **`ls`** — every parent effect is a linear shift. Each node-conditional is exactly a classical proportional-odds model. Therefore the flow, the R reference, and the truth must coincide. This variant is the clean equivalence baseline.
- **`nl`** — this variant adds an accelerating age effect on disability and a heterogeneous treatment effect `tau(Age)` that fades in the elderly. It also reduces the treatment probability for the very old. An all-`ls` model must collapse `tau(Age)` to a constant and is therefore biased. A flexible (`ci`/`cs`) flow can recover the truth. The trial cohort (`rct.csv`) enrolls younger patients than the observational cohort, as real trials do. This extrapolation is exactly what breaks the misspecified model.

The table shows the known-truth recovery (seed 7), the ATE on P(good) over the trial population:

nl variant	**ATE**	**vs true +0.104**
naive observational contrast	+0.303	confounded (overstates 2.9×)
all-`ls` flow	+0.076	undershoots (misses the age-varying effect)
flexible (`ci`/`cs`) flow	+0.101	**recovers the truth**

This result reproduces the clinical-data finding in miniature (flexible +0.108 vs all-`ls` +0.054), but against a *known* answer.

R cross-check. data/magic-mrclean/fit_ls.R refits the all-`ls` DAG node-by-node in classical R (`MASS::polr` / `tram::Colr` / `glm`). Its committed `ref_ls/` outputs let `tests/test_simulations.py` assert flow ≡ R fit (outcome-node coefficients and ATE) without an R installation.

Clinical-data results (context — not reproducible from this repo)

model	**ATE on P(good)**	**source**
MR CLEAN RCT (ground truth)	**+0.135** [+0.057, +0.213]	Berkhemer et al. 2015

model	ATE on P(good)	source
this flow, nihss6 config	**+0.063**	`experiments/stale/nihss6_flow.py` (parked)
this flow, all-ls	**+0.057**	`experiments/stale/all_ls_flow.py` (parked)
classical proportional-odds MLE, same 80% split	+0.055	`experiments/validate_ls.py`
original TRAM-DAG `md_dag_ls` (all-ls)	+0.054	paper monorepo
original TRAM-DAG `nihss6`	+0.108 (seed 2: +0.092)	paper monorepo
classical MLE, full data	+0.082	paper monorepo

Reading notes:

- When the all-ls flow trains to convergence without early stopping (the default, `restore_best=False`), **it IS the classical MLE**. On the synthetic `ls` cohort, its outcome-node coefficients match `statsmodels` *and* R `polr` to 4 decimals (Age 0.0526, NIHSSa 0.1630, T −0.9424, and ATE +0.1429 vs +0.1428). See `experiments/validate_ls.py` and `test_simulations.py::test_all_ls_flow_is_exact_mle`. The earlier residual of +0.057 vs +0.055 on the clinical data was exactly the early-stopping effect (see CHANGELOG).
- **Flexible (`ci`/`cs`) models are different**: their MLE *overfits the observational confounding*. Therefore they need `restore_best=True` (early-stopping regularization) to recover the causal effect. The synthetic `nl` cohort confirms this: the flexible MLE undershoots (+0.076), but the early-stopped fit recovers the true ATE (+0.10), with a lower validation NLL. `run_experiment` defaults `restore_best` per style accordingly (off for all-`ls`, on for flexible).
- The treatment effect is **weakly identified** in this observational cohort: the likelihood around the T-coefficient is nearly flat. Refits drift to the 80%-split optimum of ≈ +0.054. The original +0.108 is a near-likelihood-equivalent solution, not a sharper optimum. Both flow results sit inside the established acceptance band [+0.03, +0.14]. The meaningful check is the match to the band, not to the point estimate.

Counterfactual demo

`experiments/stale/counterfactual_demo.py` (parked, needs an API migration before it runs again) is a capability beyond the original scripts. It abducts the latents of the 128 held-out test patients and confirms exact factual reconstruction. It then predicts the outcome of each patient under the opposite treatment (`results/<name>/counterfactuals_test.csv`, `plots/counterfactual_mrs3m.png`).

How fast can a CausalFlowDAG train?

This document benchmarks learning-rate schedules, per-node freezing, batch sizes, devices, and LBFGS. The benchmark ran in June 2026 on an Apple-silicon Mac mini with torch 2.12 (CPU unless noted). To reproduce it, use `experiments/stale/bench_training.py` (parked). Migrate the script to the current API first. The grid takes ≈ 35 min. For a quick **cross-machine** comparison, use the self-contained `experiments/stale/perf_machine.py` (parked). It runs fixed 200-epoch workloads on all available devices and writes a machine fingerprint to JSON. After `pip install tramdag`, it runs on any machine. The raw CSV is a local artifact of the parked benchmark and is not committed.

The options, and how to use them

Schedules (`fit(..., schedule=...)`) control the learning rate over training:

value	behavior
`None` (default)	constant lr — the classic behavior, exactly as before this PR
`"plateau"`	**per-node**: a node whose own validation NLL hasn't improved by `min_delta` (default 1e-4) for `plateau_patience` epochs (default 15) gets its lr × 0.3, floored at 1e-3 × the initial lr. Each node decays independently — valid because the per-node losses have independent gradients.

`"onecycle"` and `"cosine"` were part of this benchmark and lost to `"plateau"` on every workload; 0.4.0 removed both. The result tables below keep their measured rows as the record of that decision.

Early stopping / freezing (`fit(..., freeze_patience=N)`): if the validation NLL of a node does not improve for `N` epochs, the fit *freezes* that node. A frozen node leaves the loss and the backward pass, which saves real compute. Its weights stay fixed from then on. When all nodes are frozen, the fit returns early. The fit records the freeze epochs in `flow.history["frozen"]`. Under `schedule="plateau"`, freezing also waits until the lr of the node is decayed ≥ 100×. This delay prevents a freeze while a smaller step size can still make progress. Freezing state is per-`fit()`-call: a second `fit` call trains all nodes again.

Switching it all off: for an exact comparison with classical methods (`statsmodels`, R `polr`/`tram`), omit both arguments. The benchmark changed no defaults, so

```
flow.fit(train_df, epochs=4000, learning_rate=1e-2,
         batch_size=512)  # constant lr
flow.fit(train_df, epochs=2000, learning_rate=1e-3,
         batch_size=512)  # 2nd phase
```

is still the exact-MLE path that `experiments/validate_ls.py` uses. Independent of all this, `restore_best=False` remains the default (see CHANGELOG). The guard test `tests/test_fit_schedules.py::test_plateau_freeze_preserves_exact_mle` also shows that even *with* plateau+freezing the all-`ls` fit lands on the classical MLE within the usual tolerances.

LBFGS is *not* a `fit()` option — it ships as its own method. `fit_classical` runs float64 full-batch L-BFGS with a strong-Wolfe line search and supersedes the hand-rolled recipe this report benchmarked: it is deterministic, lands on the exact MLE, matches `statsmodels`/R `polr`, and refuses non-`ls` specs (minibatch noise also regularizes the MLPs, so flexible models belong in `fit`). The float32 single-precision variant measured here was fast (< 2 s) but not robust across seeds (Findings #2) — `fit_classical`'s float64 upcast is what fixed that.

```
report = flow.fit_classical(train_df)  # exact classical MLE, seconds
```

Method: time-to-target, not loss-go-down

Each config runs once. `fit()` records per-epoch validation NLL *and* wall-clock time. From these records, we measure the seconds until the fit is within a fixed gap of a cached long-run reference (3 torch seeds, medians):

workload	model / data	reference NLL	tight tol	practical tol
stroke-ls	all-`ls` stroke DAG, frozen `magic-mrclean/ls` (n=1275, full-data MLE)	10.3042 (train)	+1e-3	+5e-3
vaca-ci	all-`ci` flow, frozen `vaca` (n=5000, 90/10 split)	4.9632 (val)	+2e-3	+1e-2

Tight ≈ exact-MLE equivalence (statsmodels/R-polr match). *Practical* ≈ coefficient-equivalent: a stroke fit with gap ≈ 3e-3 already matches the R reference coefficients within the test tolerances (`tests/test_fit_schedules.py::test_plateau_freeze_preserves_exact_mle`).

Results

stroke-ls convergence vaca-ci convergence

The table shows median seconds to target (batch 512, cpu). A "—" entry means that the fit never reached the target within the budget.

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
baseline two-phase (old default)	9.0	**21.4**	2.1	2.8[1]	no (runs 40 s / 15 s)
constant 1e-2	9.1	21.5[2]	2.2	2.8[1]	no
onecycle (1500 / 300 ep)	—	—	3.5	4.5	no
onecycle (3000 ep)	16.8	— (gap 1–2e-3)			no
cosine	—	—	2.2	3.5[1]	no
plateau + freeze	**8.9**	— (gap 2e-3)	**2.0**	2.9	**yes — 13 s / 4 s total**
LBFGS (full-batch)	**1.6** (2/3 seeds)	— (gap 4–8e-3)	n/a	n/a	yes

[1] transient: the val-NLL curve dips through the target and then drifts away. Stroke needs the 1e-3 phase to *stay*. Vaca shows mild overfitting. Final gap for the vaca baseline is 0.037. The old 520-epoch budget **underfits** vaca by ~0.03 nats. Plateau+freeze *stays* at its target. [2] constant lr at batch 512 stalls at gap 3–7e-3. Only the lr-decay phase closes the last decade. This is why the two-phase recipe existed.

Findings

1. **Per-node plateau decay + freezing is the best default-style trainer.** It has the same time-to-accuracy as the hand-tuned two-phase schedule, but it needs **no budget tuning**. It decays the lr of each node off its own validation curve. It freezes converged nodes, which is a real FLOP saving because the per-node NLLs have independent gradients. And it **stops itself**: 13 s total vs 40 s for the baseline on stroke-ls, 4 s vs 15 s on vaca-ci, at equal or better final NLL.
2. **LBFGS is spectacular but not robust.** Full-batch LBFGS reaches coefficient-level accuracy on the classical all-`ls` model in **< 2 s** (vs 9 s for Adam) on 2/3 seeds. The third seed stalls at gap 8e-3. An Adam warm start made it *worse* (different basin) on every seed. Use it as a fast first shot with the plateau trainer as fallback, not as the default.
3. **OneCycle is a "spend exactly this budget" scheduler.** Accuracy arrives only at the end of its anneal. At 1500 epochs it misses everything. At 3000 epochs it lands gap 1–2e-3. But you must know the right budget in advance, and this requirement is the problem that we try to remove.
4. **Full-batch loses on time-to-target** despite ~1.6× higher epoch throughput. It makes too few optimizer steps per second of compute at these n. Batch 512 is a good default. Very large batches (16k) only improved raw throughput at n=50k.
5. **MPS (Apple GPU) is 3–4× slower than the M-series CPU** at these model sizes (verified correct: identical reconstruction). Kernel-launch

overhead dominates sub-millisecond ops. Stay on CPU locally. CUDA on Colab-class GPUs is a different regime (see the GPU-vs-CPU race in the demo notebook).
6. **The old defaults waste or under-spend.** Stroke: 4000 epochs budgeted, converged work done after ~1500 (freezing recovers the difference automatically). Vaca: 520 epochs budgeted, ~0.03 nats short of converged. Fixed budgets are wrong in both directions. Adaptive stopping fixes both.

Recommendation

For everyday fits:

```
flow.fit(
    train,
    val,
    epochs=4000,
    learning_rate=1e-2,
    batch_size=512,
    schedule="plateau",
    plateau_patience=30,
    freeze_patience=120,
)
```

The generous `epochs` value is only a ceiling. The fit stops itself. For exact classical comparisons where the last 1e-3 matters, append a short constant-lr polish phase (`epochs=500, learning_rate=1e-3`) after the plateau fit. Or run the old two-phase recipe.

The benchmark deliberately changed no `fit()`/`run_experiment` defaults. Default changes are their own reviewed decision (see the `restore_best` episode in CHANGELOG.md). `run_experiment` still uses the two-phase constant-lr recipe; the switch to the plateau recipe remains an open 3-line follow-up.

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=)`

A `VC` term gives a node a **treatment-effect head with its own bias–variance budget** (issue #28). The term contributes

```
beta(modifiers) * x_on        with    beta(x) = beta0 + b_theta(x)
```

to the node's shift. Here `b_theta` is a deliberately small (one 16-unit hidden layer), **penalized** network. The point is not expressiveness. The point is that the flow estimates the effect function *with care*, not as a by-product:

```python
import tramdag as td

spec = {
    "X1": td.ContinuousNode(),
    "X2": td.ContinuousNode(),
    "X3": td.ContinuousNode(),
    "T": td.OrdinalNode(2, td.LS("X1") + td.LS("X2")),
    "Y": td.ContinuousNode(
        td.CS("X1", "X2", "X3")  # prognostic part g(x): as
        flexible as you like
        + td.VC("X2", "X3", t="T", penalty=1.0)  # effect
        beta(X2, X3) * T
    ),
}
flow = td.CausalFlowDAG(spec, seed=0).fit(
    train, val, epochs=500, learning_rate=1e-2, batch_size=512,
        restore_best=True
)

beta = flow.varying_coef("Y", df_new)  # (n,) array beta(x) —
        deterministic, y-free
beta0 = float(flow.nodes["Y"].shifts["T"].beta0)
        # interpretable main effect
```

`X2` and `X3` appear **twice**: as prognostic parents through `CS` and as effect modifiers through `VC`. This pattern is intended. Only the treatment node named by `t=` owns its edge, and a second term that declares it raises an error. Modifiers can repeat.

Why not `CS("T", "X2", "X3")`? (anti-pattern)

For a binary treatment, the multi-parent `CS` is equivalent in *expressiveness*. Any shift decomposes exactly as `s(x,t) = s(x,0) + [s(x,1) − s(x, 0)]·t`. But the `CS` form has **no effect-specific regularization**. The likelihood rewards a good average fit of `s(x,t)`, and nothing rewards a smooth difference between the arms. The read-out is thus the difference of two jointly fitted, unregularized networks, which amplifies noise.

On the `vc-shift` validation DGP task class, the `CS` reduced form reaches corr ≈ 0.5 against the true effect function (tramdag-simu#18 / PR #21). This holds **even when the model is exactly in-class**. The `VC` term reaches corr ≈ 0.99 on the same protocol (`tests/test_vc_term.py`, acceptance bar 0.9). Causal forests and R-learners work not because they *target* the effect, but because they **regularize** it (Nie & Wager 2021, Athey–Tibshirani–Wager 2019). `VC` brings that ingredient into the TRAM framework.

Semantics

- **Scale**: `beta(x)` lives on the node's latent (log-odds) scale. The flow adds it for a continuous node (`z = h(y) + …`) and subtracts it from the cutpoints for an ordinal node, exactly like an `LS` weight. With no modifiers, `VC(t="T")` *is* `LS("T")` (identical model, testably bit-exact). Thus `VC` versus `LS` is a nested question.
- **Penalty**: the fitting objective is the penalized likelihood Σ_i `NLL`$_i$ + `penalty` · ‖`b_theta weights`‖² on the total-NLL scale. This is a fixed Gaussian prior whose shrinkage vanishes as n grows. The penalty never applies to `beta0`. `penalty` → ∞ shrinks `b_theta` to the zero function and recovers the classical `LS` fit. The default is `penalty=1.0`. If the modifiers are many or n is small, raise the penalty.
- **Identification / centering**: a constant moves freely between `beta0` and `b_theta`. The head's output layer is zero-initialized (`beta(x) = beta0` at step 0). After `fit`, the flow re-centers the head to mean zero over the training data, which preserves the function. Thus `beta0` is the main effect in the training population, the `Colr` reading when `beta` is constant.
- **Warm start**: `fit(vc_warm_start=True)` (the default) initializes `beta0` from the classical all-`ls` solution of the node's conditional, once per term. That solution comes from a deterministic L-BFGS on a throwaway proxy. Thus training starts at the classical answer and only learns deviations.
- **Treatments**: the treatment `x_on` can be continuous or binary (2-level) ordinal. The term is linear in `x_on`. A binary ordinal enters as its 0/1 level, so `beta` is the identified level-1-vs-0 contrast. Multi-level ordinal treatments are a planned follow-up.
- **Read-out**: `flow.varying_coef(node, data, t=...)` evaluates `beta0 + b_theta(modifiers)` in closed form. The result is deterministic and y-free, and it needs no abduction. For a binary treatment, it equals the abduct-difference `u(x, t=1, y) − u(x, t=0, y)` identically (pinned by a test).

Propensity-centered VC: `center=True` (R-learner orthogonalization)

```
td.VC("X2", "X3", t="T", penalty=1.0, center=True,
        center_folds=5)
# contributes  beta(x) * (t - e_hat(x))  to the shift
```

This is Robinson/R-learner centering inside the likelihood. Dandl et al. (2024) found treatment-centering to be *the* decisive ingredient for effect estimation under confounding in model-based forests. That finding reproduces here. The test DGP is strongly confounded, and the model deliberately under-specifies its prognostic part (true g(x) quadratic, model linear). On that DGP, the uncentered β absorbs the confounded misfit. The mean $|\beta - \tau|$ is ≈ 1.1–1.2, which effectively destroys the effect estimate. The centered β stays near truth (≈ 0.1–0.3), a **5–10× bias reduction** (`tests/test_vc_centered.py`). If the prognostic part is correctly specified,

centering changes little. Centering is insurance against the misspecification that you do not know you have.

The naive implementations are wrong. Thus this is a **two-stage frozen** design:

- **Training** uses **out-of-fold** ê: the fit does `center_folds` refits of the treatment node *only*, and each refit predicts the fold that it never saw. This is the DML cross-fitting requirement. In-sample ê reintroduces the own-observation bias and can be *worse* than no centering. The OOF values enter the outcome loss as frozen data. Thus **no gradient reaches the treatment node** from the outcome node, and the per-node factorization stays intact (pinned by a gradient-isolation test). The flow exposes the fold bookkeeping in `flow.vc_center_info`, pinned by tests.
- **Inference** (`log_prob` / `sample` / `abduct` / `pmf` / `scores`) recomputes ê from the flow's **own fitted treatment node**, the full-data fit (the standard DML train/predict split). The computation is detached and uses the current parent values. Under `do(T=t)`, the flow re-derives the regressor as `t − ê(x)`. The flow caches nothing.
- **Interpretation**: with centering, `beta0` is the effect at the treatment margin (the observed propensities). `varying_coef` is unchanged, because centering moves the regressor, not β. The LS-nesting reading applies to the uncentered term only. Centering requires a binary ordinal treatment. Continuous-treatment centering with E[T|x] is a follow-up. `center=False` (the default) is bit-identical to the uncentered term.

Validation

`tramdag.simulations.VCLogisticShift` (`data/vc-shift/`, frozen contract) is a logistic-shift SCM with known `beta_true(x) = −1 + 0.8·X2 − 0.6·X3`. It has a nonlinear prognostic part and confounded assignment (X2 is confounder *and* modifier). Acceptance (`tests/test_vc_term.py`) requires three results: recovery corr ≥ 0.9 at n = 5000 (measured ≈ 0.99, min over 3 seeds 0.986), a fitted `beta0` that matches `fit_classical` under a large penalty, and the read-out identities. Candidate follow-ups are separate issues: propensity-centered `beta(x)·(t − ê(x))` (#30), and per-observation scores for effect-modifier scans (#29).